

MODÜL 05

Regularization

Overfitting, Dropout, Batch Norm, Early Stopping

L1·L2 · Dropout · Batch Norm · Layer Norm · Early Stopping · Data Augmentation

Seviye	Orta — Uygulamalı
Kapsam	Overfitting · L1/L2 · Dropout · BN · LN · Early Stopping · Augmentation
Framework	NumPy · TensorFlow/Keras · PyTorch
Önceki Modül	Modül 04 — Backpropagation
Sonraki Modül	Modül 06 — CNN (Evriimsel Sinir Ağları)

Modül Konsept Haritası

Overfitting Nedir?		Tanı		Çözüm Araçları
Bias-Variance		Train↓ Test↑		Regularization
Ağırlık Cezası		Yapısal		Eğitim Stratejisi
L1 · L2		Dropout · BN · LN		Early Stop · Augment
NumPy Impl.		Framework Kullanımı		TF · PyTorch

1. Öğrenme Hedefleri

- ✓ Bias-variance tradeoff'u matematiksel olarak ifade etmek ve grafik üzerinde göstermek
- ✓ L1 ve L2 regularization gradyanlarını türetmek ve ağırlıklara etkisini karşılaştırmak
- ✓ Inverted Dropout'u eğitim ve inference modunda doğru uygulamak
- ✓ Batch Normalization algoritmasının 4 adımını açıklamak ve sıfırdan uygulamak
- ✓ Layer Norm ile Batch Norm arasındaki farkı Transformer bağlamında açıklamak
- ✓ Early stopping'i validation loss takibiyle uygulamak ve en iyi modeli saklamak
- ✓ Veri artırma (data augmentation) tekniklerini görüntü ve metin için listelemek
- ✓ TF/Keras ve PyTorch'ta tüm regularization tekniklerini kodlamak

2. Overfitting vs Underfitting

Overfitting: model eğitim verisini ezberler, genelleme yapamaz. Train hatası düşer ama test hatası yükselir. Temel ayrım bias-variance tradeoff ile formüle edilir:

$$\text{Toplam Hata} = \text{Bias}^2 + \text{Variance} + \text{Gürültü}$$

Durum	Bias	Variance	Eğitim Hatası	Test Hatası	Çözüm
Underfitting	Yüksek	Düşük	Yüksek	Yüksek	Daha derin model
İyi Fit	Orta	Orta	Düşük	Düşük	—
Overfitting	Düşük	Yüksek	Çok Düşük	Yüksek	Regularization!

Polinom fit örneği: Derece 1 underfitting (test MSE=0.20), Derece 4 iyi fit (test MSE=0.03), Derece 20 overfitting (test MSE=0.42).

3. L1 ve L2 Regularization

Temel fikir: Kayıp fonksiyonuna ağırlıkların büyüklüğünü cezalandıran bir terim ekle. Böylece model basit ağırlıklar öğrenir ve genelleme artar.

3.1 L2 Regularization (Ridge / Weight Decay)

$$L_{\text{total}} = L_{\text{data}} + (\lambda/2) \cdot \sum w_i^2$$

- Türev: $\partial L/\partial w = \partial L_{\text{data}}/\partial w + \lambda \cdot w$
- Güncelleme: $w \leftarrow w \cdot (1 - \eta \cdot \lambda) - \eta \cdot \partial L_{\text{data}}/\partial w$
- Etki: her ağırlık her adımda $(1 - \eta \cdot \lambda)$ oranında küçülür
- Sıfıra yaklaşır ama SIFIR OLMAZ (üstel azalma)
- Kullanım: genel regularization — varsayılan tercih

3.2 L1 Regularization (Lasso)

$$L_{\text{total}} = L_{\text{data}} + \lambda \cdot \sum |w_i|$$

- Türev: $\partial L/\partial w = \partial L_{\text{data}}/\partial w + \lambda \cdot \text{sign}(w)$
- Güncelleme: $w \leftarrow w - \eta \cdot \lambda \cdot \text{sign}(w) - \eta \cdot \partial L_{\text{data}}/\partial w$
- Etki: küçük ağırlıkları tam SIFIR yapar → seyrek model
- Sıfırlama nedeni: $\text{sign}(w)$ sabit büyüklükte iter, küçük w sıfırı geçer
- Kullanım: özellik seçimi, yorumlanabilir modeller

Özellik	L2 (Ridge)	L1 (Lasso)
Ceza terimi	$(\lambda/2) \cdot \sum w^2$	$\lambda \cdot \sum w $
Gradyan	$\lambda \cdot w$ (w ile orantılı)	$\lambda \cdot \text{sign}(w)$ (sabit)
Ağırlık davranışı	Sıfıra yaklaşır	Tam sıfır olabilir
Model özelliği	Tüm özellikler kalır	Bazı özellikler elenir
En iyi durum	Genel regularization	Özellik seçimi, seyreklik
PyTorch kullanımı	weight_decay parametresi	Manuel loss terimi

4. Dropout

Dropout (Srivastava et al., 2014): eğitim sırasında her nöronu p olasılıkla rastgele devre dışı bırak. Her iterasyonda farklı bir 'alt ağ' eğitilir. Bu, birçok farklı ağın ensemble'ına benzer bir etki yaratır.

Inverted Dropout — Formül:

$\text{mask} = \text{Bernoulli}(1-p) \rightarrow 0 \text{ veya } 1 \text{ değerler, } p \text{ olasılıkla } 0$

$a_{\text{train}} = a * \text{mask} / (1-p) \leftarrow \text{beklentiyi korumak için } (1-p)'ye \text{ böl!}$

$a_{\text{infer}} = a \leftarrow \text{inference'da değişiklik yok}$

Neden $(1-p)$ 'ye bölünür?

Eğitimde n nörondan ortalama $n \cdot (1-p)$ tanesi aktif. Inference'da hepsi aktif $\rightarrow$ toplam aktivasyon $(1-p)$ kat büyür. Inverted dropout bu farkı eğitim sırasında dengeleyerek inference'ı değiştirmeden bırakır.

Özellik	Eğitim Modu	Inference Modu
Mask	Bernoulli($1-p$) — her batch farklı	Yok
Ölçekleme	$a / (1-p)$	a (değişmez)
$p=0.5$ ile	Ortalama $n/2$ nöron aktif	Tüm n nöron aktif
Amaç	Co-adaptation önle, gürültü ekle	Deterministik tahmin
Tipik p değ.	0.5 (gizli katman)	0.1-0.3 (giriş katmanı)

5. Batch Normalization

Batch Normalization (Ioffe & Szegedy, 2015): her mini-batch içinde aktivasyonları normalize et. Derin ağlarda iç katmanların dağılımı sürekli değişir (internal covariate shift) — BN bunu önler.

Algoritma (4 Adım):

Adım	İşlem	Formül
1	Batch ortalamasını hesapla	$\mu_B = (1/m) \cdot \sum x_i$
2	Batch varyansını hesapla	$\sigma^2_B = (1/m) \cdot \sum (x_i - \mu_B)^2$
3	Normalize et	$\hat{x}_i = (x_i - \mu_B) / \sqrt{(\sigma^2_B + \epsilon)}$
4	Ölçek ve kaydır (γ, β öğrenilir)	$y_i = \gamma \cdot \hat{x}_i + \beta$

- γ ve β : ağ tarafından öğrenilen parametreler — BN'nin tamamen sıfırlamamasını sağlar
- $\epsilon = 1e-5$: sıfıra bölme önlemi
- Inference: hareketli ortalama (running mean/var) kullanılır, batch istatistikleri değil
- Avantaj: büyük lr kullanılabilir, başlatmaya daha az hassas, regularization etkisi

6. Layer Normalization

Layer Normalization (Ba et al., 2016): Batch Norm'dan farkı — normalize etme yönü. BN her özelliği batch boyunca normalize ederken, LN her örneği kendi özellikleri boyunca normalize eder.

Özellik	Batch Normalization	Layer Normalization
Normalize yönü	Her özellik → batch boyunca	Her örnek → özellikler boyunca
Batch bağımlılığı	Evet — batch boyutu önemli	Hayır — her örnek bağımsız
Sıralı veri (NLP)	Sorunlu (değişken uzunluk)	İdeal
Inference farkı	Hareketli ortalama gerekir	Her zaman aynı davranış
Kullanım	CNN, görüntü modelleri	Transformer, BERT, GPT, ViT
Formül	μ, σ^2 batch boyunca	μ, σ^2 özellikler boyunca

7. Early Stopping

Validation loss iyileşmemeye başladığında eğitimi durdur. Overfitting'in en basit ve etkili çözümlerinden biridir. En iyi modelin ağırlıkları saklanır ve eğitim sonunda geri yüklenir.

- patience: kaç epoch boyunca iyileşme olmazsa eğitim durdurulur (tipik: 5-20)
- delta: 'iyileşme' sayılması için gereken minimum azalma (tipik: 1e-4)
- restore_best_weights: en iyi epoch'taki ağırlıkları geri yükle
- monitor: genellikle 'val_loss', bazen 'val_accuracy'

8. Data Augmentation

Teknik	Açıklama	Kullanım
Yatay Flip	Görüntüyü yatay çevir	CIFAR, ImageNet
Rastgele Kırpma	Görüntüden rastgele bölge al	Her CV görevi
Döndürme	$\pm 15-30$ derece	Nesne tanıma
Color Jitter	Parlaklık/kontrast/doygunluk değiştir	CV genel
Cutout / RandomErase	Rastgele bölgeyi siyahla	CIFAR-10/100
Mixup	İki görüntüyü ve etiketi karıştır	İnce ayar
CutMix	Bir görüntünün parçasını diğerine yap.	ImageNet SOTA
Back Translation	Farklı dile çevir ve geri çevir	NLP veri artırma
Synonym Replace	Kelimeleri eş anlamıyla değiştir	Metin sınıflandırma

9. Framework Karşılaştırması

Teknik	TensorFlow / Keras	PyTorch
L2	Dense(64, kernel_regularizer=l2(0.01))	optim.Adam(..., weight_decay=1e-4)
L1	Dense(64, kernel_regularizer=l1(0.01))	loss += lambda*sum(p.abs().sum())
Dropout	layers.Dropout(rate=0.5)	nn.Dropout(p=0.5)
Batch Norm 1D	layers.BatchNormalization()	nn.BatchNorm1d(num_features)
Batch Norm 2D	layers.BatchNormalization()	nn.BatchNorm2d(num_channels)
Layer Norm	layers.LayerNormalization()	nn.LayerNorm(normalized_shape)
Early Stop	callbacks.EarlyStopping(patience=10)	Manuel val_loss takibi
Train/Eval mod	model.fit() otomatik yönetir	model.train() / model.eval()

10. Yaygın Hatalar ve Tuzaklar

Hata 1: Dropout'u inference'da kapatmayı unutmak (PyTorch)	<code>model.eval()</code> çağrılmazsa Dropout inference'da da aktif kalır — her tahmin farklı sonuç verir. Her zaman <code>model.eval()</code> kullan.
Hata 2: Batch Norm'un train/eval modunu değiştirmemek	BN eğitimde batch istatistiklerini, inference'da hareketli ortalamaları kullanır. <code>model.eval()</code> ile modu değiştirmezsen tutarsız sonuçlar alırsın.
Hata 3: L2 ve <code>weight_decay</code>'i ayrı uygulamak	PyTorch'ta AdamW, L2'yi gradyan ölçeklenmesinden bağımsız uygular (Loshchilov & Hutter, 2019). Adam + <code>weight_decay</code> yerine AdamW tercih et.
Hata 4: Çok büyük dropout oranı seçmek	$p=0.8$ gibi yüksek değerler modeli çok kısıtlar — underfitting'e yol açar. Gizli katmanlar için $p=0.3-0.5$ tipik başlangıç.
Hata 5: Early stopping'de <code>restore_best_weights</code>'i kullanmamak	Early stop tetiklendiğinde model son epoch ağırlıklarıyla durur, en iyi ağırlıklarla değil. <code>restore_best_weights=True</code> ekle.
Hata 6: Batch Norm öncesi/sonrası sırası	BN'nin aktivasyondan önce mi (pre-activation) yoksa sonra mı (post-activation) uygulanacağı tartışmalıdır. ResNet v2 pre-activation öneriyor; pratik başlangıç: aktivasyon sonrası.

11. Özet Tablo

Teknik	Ne Yapar	Ne Zaman Kullan
L2 (Weight Decay)	Ağırlıkları küçültür, sıfırlamaz	Her zaman — varsayılan tercih
L1 (Lasso)	Bazı ağırlıkları sıfıra iter	Özellik seçimi, seyrek model
Dropout	Rastgele nöron kapat	Tam bağlantılı katmanlar
Batch Norm	Batch içi aktivasyon normalize	CNN — her Conv sonrası
Layer Norm	Örnek kendi içi normalize	Transformer, RNN, NLP
Early Stopping	Val loss artınca dur	Her uzun eğitimde
Data Augmentation	Veri çeşitliliğini artır	Az veri, görüntü/metin görevleri

Hızlı Rehber:

- Overfitting varsa → Dropout + L2 + Early Stopping
- Derin CNN eğitiyorsan → Batch Norm (her Conv sonrası) + Dropout (FC katmanlar)
- Transformer yazıyorsan → Layer Norm + Dropout
- Az verin varsa → Data Augmentation + Transfer Learning

12. Kaynaklar

- [1] Srivastava, N. et al. (2014). Dropout: A Simple Way to Prevent Neural Networks from Overfitting. JMLR.
- [2] Ioffe, S. & Szegedy, C. (2015). Batch Normalization: Accelerating Deep Network Training. ICML.
- [3] Ba, J.L. et al. (2016). Layer Normalization. arXiv:1607.06450.
- [4] Loshchilov, I. & Hutter, F. (2019). Decoupled Weight Decay Regularization (AdamW). ICLR.
- [5] DeVries, T. & Taylor, G.W. (2017). Improved Regularization of Convolutional Neural Networks with Cutout. arXiv.
- [6] Zhang, H. et al. (2018). mixup: Beyond Empirical Risk Minimization. ICLR.
- [7] Goodfellow, I., Bengio, Y. & Courville, A. (2016). Deep Learning, MIT Press. Bölüm 7.

Sonraki Adım: Modül 06 — CNN | Konvolüsyon • Pooling • ResNet • EfficientNet • Grad-CAM